wsdm-cup-multilingual-chatbot-arena

7位 Solution

Rank	7
Team	Team Turing
Author	Raja Biswas
Topic ID	567589
Version	Japanese Translation

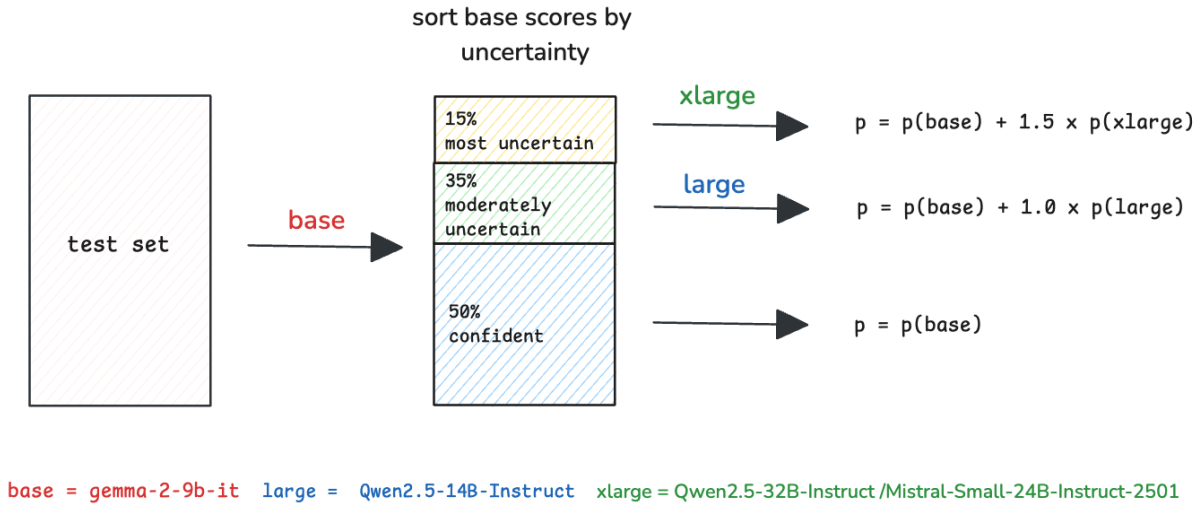
Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567589
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Kaggle、コンペの主催者、そして参加者の皆さん、このコンペをエキサイティングなものにしてくださりありがとうございます。上位のsolutionがテストセットにうまく汎化しているのを見られて嬉しく思います。入賞された皆さん、おめでとうございます！Trushantと私は、3週間のsprintでsolutionを作り上げるのをとても楽しみました。

推論パイプライン

下図のように、不確実性に基づくensembleパイプラインを実装しました。



- まず、fine-tuneした `gemma-2-9b-it` modelで全テスト例をscoringしました
- 次に、決定境界への近さとして不確実性scoreを計算しました（`uncertainty = 1-|p-0.5|`）
- 不確実性が上位50%の例（すなわち確率が0.5に最も近い例）を特定し、より大きなmodelで改めてscoringしました。

具体的には、

- 不確実性が上位15%の例には、fine-tuneした `Qwen2.5-32B-Instruct`（選択した提出1）または `Mistral-Small-24B-Instruct-2501`（選択した提出2）model（xlarge）を使いました
- 次の35%の例には、`Qwen2.5-14B-Instruct`（large）を使いました
- 残りの例には、`gemma-2-9b-it` model（base）のscoreをそのまま使いました

最終scoreは次のensembleで計算しました。

- 不確実性上位15%の例: `Score = 1.0 x base + 1.5 x xlarge`
- 次の35%の例: `Score = 1.0 x base + 1.0 x large`
- 残りの例: `Score = 1.0 x base`

推論にはvllmを使いました。512例あたりの推論時間の見積もりを、max sequence lengthとともに下に示します。

```
||||
||||
||||
||||
||||
||||
```

データ

学習には次のデータソースを使いました。

ラベル付きDataset

- WSDM Cup - Multilingual Chatbot Arenaコンペのデータ（45k）
- LMSYS - Chatbot Arena Human Preference Predictionsコンペのデータ（30k）
- lmarena-ai/PPE-Human-Preference-V1（10k）

ラベルなしDataset

- wsdm - open models - nbroad（NicholasのDataset - 6kをサンプリング）
- Skywork/Skywork-Reward-Preference-80K-v0.2（25kをサンプリング）
- allenai/tulu-3-sft-olmo-2-mixture（30kをサンプリング）
- チームで生成した好みのペア（10万例）
  - allenai/WildChat-1M と lmsys/lmsys-chat-1m datasetのprompt
  - Arena Explorer の分布におおよそ合わせるための、クラスタに基づくサンプリング

Model

Gemma、Llama、Qwen、Mistralの各familyのmodelをfine-tuneし、次のscoreを得ました。

Model	Dataset	#	Validation Accuracy	Validation AUC	Public LB
gemma-2-2b-it	Comp + 10k Labelled + 80k Pseudo Labelled		0.676	0.742	0.675
gemma-2-9b-it	Comp + 30k Labelled + 120k Pseudo Labelled		0.702	0.777	0.696
Qwen2.5-14B-Instruct	Comp + 30k Labelled + 120k Pseudo Labelled		0.699	0.777	0.696
Qwen2.5-32B-Instruct	Comp + 30k Labelled + 120k Pseudo Labelled		0.701	0.780	x
Mistral-Small-24B-Instruct-2501	Comp + 30k Labelled + 120k Pseudo Labelled		**0.704**	**0.783**	x
Qwen2.5-72B-Instruct	Comp		0.704	0.779	x
Llama-3.1-Nemotron-70B-Instruct	Comp		0.693	0.770	x

- まず `Qwen2.5-72B-Instruct` と `Llama-3.1-Nemotron-70B-Instruct` をコンペのsubsetでfine-tuneし、ラベルなしdatasetの擬似ラベル生成に使いました。
- Gemma modelでは、"A" と "B" tokenのlogitを直接使って好みの確率を計算しました。これらのmodelにはreward model headがありません。language modelingとreward modelingのlossを組み合わせて学習しました。（参考: Regularizing Hidden States Enables Learning Generalizable Reward Model for LLMs）

- 残りのmodelは、linear headを持つ標準の `AutoModelForSequenceClassification` architectureで学習しました。

汎化

汎化を改善するため、次の点を考慮しました。

- 不確実な例を扱うために、より大きなLLMを推論パイプラインに組み込みました。大きなLLMほど汎化しやすい傾向があります。
- 異なるmodelingのアプローチとmodel family（logitベースとAutoModelForSequenceClassification）のensemble。
- 好みをmodelingするheadをシンプル（linear layer 1層）に保ちました。
- 多様なpromptを含む大規模なラベルなしdatasetへの擬似ラベルを使った、70B+ LLMからのdistillation。

リンク

- 推論Notebook

皆さんのsolutionを読んで学ぶのを楽しみにしています。完璧なコラボレーションとチームワークをしてくれた @trushk に特別な感謝を！

補足Q&A

Cody_Null — 質問

素晴らしい成果ですね！私たちもこういうことをやりたかったのですが、残念ながらGemmaでvllmを動かせず、qwen modelでも最初は良い結果が出なかったので、早々に見切りをつけてしまったようです。これらのmodelをどうやってvllmで動かしたのか、ぜひ詳しく聞きたいです。

Raja Biswas（著者） — 回答

@cody11null さん、ありがとうございます！

Gemmaでは、学習と推論の両方でlogit soft-cappingを無効にする必要がありました（そうしないと、soft probのscoreを得るのに苦労しました）。はい、Qwen modelはGemmaと本当にうまくblendできました。vllm推論については、私たちの推論notebookを参照してください。

ducnh279 — 質問

@conjuring92 さんとチームの皆さん、おめでとうございます！

参考論文を読んだところ、彼らは**language modeling head（`lm_head`）を残したまま、別のreward headを追加**し、language modelingとreward modelingの両方のlossで学習していました。しかし、あなたのアプローチは**新しいreward headを導入せず**、`lm_head` に頼って特定のpooling token（'A', 'B'）のlogitを使っています。

この選択の理由を教えていただけますか。両方のアプローチを試して、`lm_head` を使う方が同等の性能を保ちつつ効率的だと分かったのでしょうか。

私の経験では、decoder modelの `lm_head` をfine-tuneする場合、パラメータが多く既に事前学習されているため、通常は**backboneより小さいlearning rate**が必要です。これは、分類headをbackboneより**大きい**learning rateで学習することが多いencoder modelとは対照的です。私の場合、`lm_head` をembedding次元と同程度のサイズの**新しいheadに置き換える**ことがよくあります。これでパラメータ数が大幅に減り、encoder modelのfine-tuningと同様の、より標準的なlearning rateを使えることが分かっています。

ちなみに、あなたのアプローチは、汎化を高めるreward modelを学習するための興味深く価値ある手法です。この論文 😂 の手法、"Fine-Tuning Can Distort Pretrained Features and Underperform Out-of-Distribution." を思い出しました。特に、最後のlinear layerだけを学習するLinear Probing（LP）と、LPの後にmodel全体をfine-tuneするLP-FTは、汎化に効果的であることが示されています。

Raja Biswas（著者） — 回答

@ducnh279 さん、洞察に富んだコメントと経験の共有をありがとうございます！

> 私の経験では、decoder modelのlm_headをfine-tuneする場合、パラメータが多く既に事前学習されているため、通常はbackboneより小さいlearning rateが必要です。これは、分類headをbackboneより大きいlearning rateで学習することが多いencoder modelとは対照的です。

私も同じ経験があります。通常、`lm_head` には他のparameter groupより10倍低いlrを使います。

> 参考論文を読んだところ、彼らはlanguage modeling head（lm_head）を残したまま、別のreward headを追加し、language modelingとreward modelingの両方のlossで学習していました。しかし、あなたのアプローチは新しいreward headを導入せず、lm_headに頼って特定のpooling token（'A', 'B'）のlogitを使っています。

はい、鋭い指摘です。実装をシンプルにするため、そしてさらに強い正則化の効果があるはずだと想定して、headを1つにしました。論文で提案された元のアプローチとの性能比較はできていません。

> ちなみに、あなたのアプローチは、汎化を高めるreward modelを学習するための興味深く価値ある手法です。この論文 😂 の手法、"Fine-Tuning Can Distort Pretrained Features and Underperform Out-of-Distribution." を思い出しました。特に、最後のlinear layerだけを学習するLinear Probing（LP）と、LPの後にmodel全体をfine-tuneするLP-FTは、汎化に効果的であることが示されています。

良い論文ですね！紹介してくださりありがとうございます。大きな分布シフトが予想される場合は、最後の層だけをfine-tuneする方が良いかもしれません。そうでなければ、modelの学習が不十分になるかもしれません。